

Process Verification

Lab 5

Quentin Meneghini

5th May 2026

Facultat d'Informàtica de Barcelona

Contents

1	Design Overview	1
1.1	FIFO	1
1.1.1	Parameters	1
1.1.2	Interface	1
1.1.3	Functional Description	2
1.2	CACHE	2
1.2.1	Parameters	2
1.2.2	Interface	3
1.2.3	Functional Description	3
2	Tasks	5
2.1	Task 1 - FIFO	5
2.2	Task 2 - FIFO	6
2.3	Task 3 - FIFO	6
2.4	Task 4 - CACHE	7
2.5	Task 5 - CACHE	8
A	Appendix Chapter	9
A.1	Use of tools	9
A.2	Use of time	9
A.3	Use of AI	9

Chapter 1

Design Overview

There are 2 modules under test: FIFO and cache.

1.1 FIFO

A circular buffer. It utilizes internal write and read pointers to manage data flow and provides status flags to prevent data loss or invalid reads.

1.1.1 Parameters

There are two parameters to this dut:

- WIDTH : data size.
- DEPTH : fifo depth.

1.1.2 Interface

Signal	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low synchronous reset
wr_en	Input	1	Write enable signal
rd_en	Input	1	Read enable signal
wr_data	Input	WIDTH	Write data
rd_data	Output	WIDTH	Read data
full	Output	1	Full signal
empty	Output	1	Empty signal

Table 1.1: Fifo signals

1.1.3 Functional Description

- **Reset State:** On a low-level reset (`rst_n`), all internal pointers and the occupancy count are cleared. The empty flag is asserted, and the full flag is de-asserted.
- **Write Operation:** Triggered by `wr_en`. If the FIFO is not full and no simultaneous read is requested, data is written to the current `wr_ptr` location, the pointer increments (modulo `DEPTH`), and the count increases.
- **Read Operation:** Triggered by `rd_en`. If the FIFO is not empty and no simultaneous write is requested, data is retrieved from `rd_ptr`, the pointer increments (modulo `DEPTH`), and the count decreases.
- **Conflict Resolution:** The logic includes a strict mutual exclusion rule: if both `wr_en` and `rd_en` are high, the FIFO performs no action, maintaining the current state.
- **Flag Logic:**
 - **Full:** Asserted when `count == DEPTH`.
 - **Empty:** Asserted when `count == 0`.

1.2 CACHE

A direct-mapped cache. It translates an 8-bit address into Tag, Index, and Offset fields to determine if requested data is currently "local" (hit) or needs to be fetched (miss).

1.2.1 Parameters

There are four parameters to this dut:

- `CACHE_LINES` : lines in the cache.
- `LINE_SIZE` : size of each line.
- `ADDR_WIDTH` : size of the address.
- `DATA_WIDTH` : size of the data.

Signal	Direction	Width	Description
clk	Input	1	Clock signal
reset	Input	1	synchronous reset
read	Input	1	read signal
write	Input	1	write signal
addr	Input	ADDR_WIDTH	address
data_in	Input	DATA_WIDTH	input data
data_out	Output	DATA_WIDTH	output data
hit	Output	1	hit signal

Table 1.2: Cache signals

1.2.2 Interface

1.2.3 Functional Description

- **Address Decoding:** The input address (`addr`) is partitioned into three functional fields:
 - **Tag:** Identifies the memory block.
 - **Index:** Selects one of the 16 cache lines.
 - **Offset:** Selects the specific 32-bit word within the 4-word line.
- **Read Operation (Priority):**
 - **Hit:** If the indexed line is valid and the stored tag matches the request, the `hit` signal is asserted and `data_out` is driven from internal memory.
 - **Miss:** If the tag differs or the line is invalid, `hit` is de-asserted. The cache performs a simulated fetch, updating the `cache_mem`, `tag_array`, and `valid_array` with the new data.
- **Write Operation:** Updates the `cache_mem` with `data_in`.
 - If the access matches an existing valid tag, it is reported as a `hit`.
 - If the access is a `miss`, the `tag_array` and `valid_array` are updated to reflect the new memory block.
- **Precedence Rule:** The module implements a strict priority for read requests. If both `read` and `write` signals are asserted simultaneously, the design executes the read operation only, and the write request is ignored.
- **Replacement Policy:** Being a direct-mapped cache, it utilizes an implicit "always over-

write” policy. Any access to an occupied index with a non-matching tag results in the previous data being replaced without further logic.

Chapter 2

Tasks

2.1 Task 1 - FIFO

There are 13 covergroup bins that are written in the `covergroup fifo_cov` block. The Table 2.1 describe better the various bins.

The directive is explicit in the property block `becomes_empty_after_read`.

Coverpoint	Bins	Names
<code>full</code>	1	<code>went_full</code>
<code>empty</code>	1	<code>went_empty</code>
<code>dut.count</code>	3	<code>empty_bin</code> , <code>mid_bins</code> , <code>full_bin</code>
<code>wr_en</code>	2	0 and 1 (Implicit)
<code>rd_en</code>	2	0 and 1 (Implicit)
Cross <code>wr_en</code> , <code>rd_en</code>	4	<code>write_only</code> , <code>read_only</code> , <code>both_off</code> , <code>illegal</code>
Total Bins	13	

Table 2.1: FIFO bins

Using only the reset sequence we cover only partially the dut. 5 bins were covered out of 13 and 0 directives out of 1 directive. In particular each uncovered bin is listed in Table 2.2.

Covergroup Object	Specific Bin	Functional Justification
full	went_full	the buffer was never full
dut.count	mid_bins	the buffer was always empty
dut.count	full_bin	the buffer was never full
wr_en	1	the FIFO is never written
rd_en	1	the FIFO is never read
cross wr_en, rd_en	write_only	the FIFO is never written
cross wr_en, rd_en	read_only	the FIFO is never read
cross wr_en, rd_en	illegal	wr_en, rd_en were never 1 at the same time

Table 2.2: Coverage Miss during Reset Sequence

2.2 Task 2 - FIFO

Using the full sequence we cover only partially the dut. 10 bins were covered out of 13 and 0 directives out of 1 directive. In particular each uncovered bin is listed in Table 2.3.

Covergroup Object	Specific Bin	Functional Justification
full	went_full	the buffer was never full
dut.count	full_bin	the buffer was never full
cross wr_en, rd_en	illegal	wr_en, rd_en were never 1 at the same time

Table 2.3: Coverage Miss during enabled full Sequence

2.3 Task 3 - FIFO

To reach 100% coverage of the dut the FIFO was first filled with 1s. Tried to write when full (17 > 16). Then valued the edge cases of cross wr_en, rd_en. At this point the buffer was made empty and refilled with 0s to toggle all bits of the internal memory. Read back. The final stimulus is to toggle the reset bit first to 0 and then to 1.

```

45  @(posedge clk);
46
47  wr_en = 1; rd_en = 0; wr_data = 8'hFF; repeat (17) @(posedge clk);
48
49  wr_en = 0; rd_en = 0;                                     @(posedge clk);
50  wr_en = 1; rd_en = 1;                                     @(posedge clk);
51
52  wr_en = 0; rd_en = 1;                                     repeat (16) @(posedge clk);
53  wr_en = 1; rd_en = 0; wr_data = 8'h00; repeat (16) @(posedge clk);
54  wr_en = 0; rd_en = 1;                                     repeat (16) @(posedge clk);
55
56  rst_n = 0;                                               @(posedge clk);
57  rst_n = 1;                                               @(posedge clk);

```

2.4 Task 4 - CACHE

A covergroup was used to cover the main points of the dut, a property covered the reset sequence. Coverage over hit/miss (and code toggling) was also used to check indirectly the `tag`. A directive is used to cover reset.

- **Operations:** read/write/idle.
- **Outcomes:** hit/miss.
- **Cross Coverage:** read/write * hit/miss.
- **Structure:** indices + offsets.
- **Corner Case:** read + write.

The signals `read`, `write`, and `addr` have been propagated to be able to confront input and sequential output in the covergroup. The resulting signals are: named adding the suffix `_past` to the original name. After executing the random stimulus already provided, we obtained a 90.47% weighted coverage and a 92.59% absolute coverage, while the directive is fully covered. In fact 2 bins out of 27 are not covered: the edge case with the collision between the signals `read` and `write` (both in case of hit-and-miss).

```

69  covergroup simple_cache_cov @(posedge clk iff !reset);
70      option.per_instance = 1;
71      option.name = "simple_cache_cov";
72
73      cp_index:  coverpoint addr_past[
74          dut.OFFSET_WIDTH + dut.INDEX_WIDTH-1 -: dut.INDEX_WIDTH];
75      cp_offset: coverpoint addr_past[
76          dut.OFFSET_WIDTH -1 -: dut.OFFSET_WIDTH];
77
78      cp_rw_hit: coverpoint {read_past, write_past, hit} {
79
80          // 3'b[read][write][hit]
81          bins read_hit      = {3'b101};
82          bins read_miss     = {3'b100};
83
84          bins write_hit     = {3'b011};
85          bins write_miss    = {3'b010};
86
87          bins collision_hit  = {3'b111};
88          bins collision_miss = {3'b110};
89
90          bins idle          = {3'b000};
91          illegal_bins fake   = {3'b001};
92      }
93
94  endgroup
95
96  property reset_p;
97      @(posedge clk)

```

```
98     reset | => (hit == 1'b0) && (dut.valid_array == '{default:0});  
99     endproperty
```

2.5 Task 5 - CACHE

To reach the full coverage of the dut, the addition of a simple stimulus to target the edge cases was enough. The added stimulus fills a random address and then tries to read it. First with the same tag, and then with a different one to get both hit-and-miss. Furthermore, the toggle or the reset signal was added to reach 100% toggle coverage and overall coverage.

```
69     repeat(5) begin  
70         @(posedge clk);  
71         addr = $urandom_range(0,255);  
72         read = 0; write = 1;  
73         @(posedge clk);  
74         read = 1; write = 1;  
75         @(posedge clk);  
76         addr[dut.ADDR_WIDTH-1 -: dut.TAG_WIDTH] = ~addr[dut.ADDR_WIDTH-1 -:  
77             dut.TAG_WIDTH];  
77         read = 1; write = 1;  
78     end  
79  
80     repeat(2) @(posedge clk);  
81     reset = 1;  
82     read = 0; write = 0;  
83     @(posedge clk);  
84     reset = 0;
```

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of 6 h distributed as follows.

- Understanding the problem: 1h.
- Fifo: 1h.
- Cache: 3h.
- Report: 1h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer